

Doctor Agent OS

Updated strategy report for the Open Prior Auth Workbench

From standards-shaped synthetic workbench to full-fledged agentic healthcare admin system

Updated May 22, 2026

New positioning: an open-source, synthetic-data-only agentic operating system for healthcare administrative workflows, starting with prior authorization as the flagship demo. The project builds the agent runtime, tool runtime, MCP server, workflow engine, standards-shaped healthcare adapters, evaluation harness, and agent cockpit from scratch while preserving honest non-production, non-PHI, non-certified boundaries.

Decision area	Updated answer
Project identity	Doctor Agent OS, with Open Prior Auth Agent Workbench as the first product application.
Near-term build target	A supervised multi-agent prior authorization case run: intake -> requirements -> DTR-style documentation -> evidence mapping -> human approval -> PAS-style packet -> mock payer response -> audit trace.
Actual production north star	Real EHR launch, payer endpoint discovery, production FHIR data plane, real security/authz/audit, durable workflow execution, integration pilots, and formal test-kit/conformance path.
What changes now	The system is no longer only a workbench. It becomes an agentic OS with custom runtime, custom tool broker, MCP boundary, healthcare standards gateway, policy gates, and evaluations.
Honest claim language	Agentic, standards-shaped, synthetic-only, test-path aware. Not production healthcare infrastructure, not certified, not PHI-ready, not a real payer connector.

This document should become the new strategic source of truth for README positioning, roadmap planning, repository structure, demo narrative, milestone scoping, and architecture docs. It supersedes the prior revised strategy report only in direction: it keeps the same honesty around synthetic data and non-conformance, but upgrades the build target into a full agentic system.

Table of contents

- 1. Executive summary
- 2. What changed from the previous strategy
- 3. Revised goal, scope, and north-star separation
- 4. Product identity and recommendation
- 5. Agentic PRD
- 6. Users, personas, and agent-facing jobs to be done
- 7. Agent system design
- 8. Doctor Runtime - custom agent runtime
- 9. Doctor ToolNet - custom tool runtime
- 10. Doctor MCP - external agent interoperability boundary
- 11. Standards gateway and conformance honesty
- 12. End-to-end agentic workflow design
- 13. Agent cockpit UI and demo narrative
- 14. Reference architecture and monorepo structure
- 15. Core data model
- 16. API, tool, and MCP catalog
- 17. Evaluation harness and golden traces
- 18. Safety, policy, and governance model
- 19. Technical stack and build sequencing
- 20. Roadmap and delivery plan
- 21. Definition of done
- 22. Production readiness gap map
- 23. Risks and mitigation plan
- 24. Repository documentation and external-facing artifacts
- 25. Final recommendation
- 26. References

1. Executive summary

The current Open Prior Auth Workbench strategy was directionally strong: a synthetic, open-source, provider-side prior authorization reference workbench that demonstrates CRD-style requirement discovery, DTR-style documentation capture, PAS-style packet assembly, payer determination tracking, operational queues, and a clear production gap map. The updated direction keeps that foundation but changes the center of gravity.

The new product should become Doctor Agent OS: an open-source agentic healthcare admin system, starting with prior authorization as the first deep workflow. The project should no longer be framed only as a FHIR-first workbench. It should be framed as an agentic system that can coordinate long-running healthcare administrative work through controlled agents, auditable tools, standards-shaped adapters, human approvals, and deterministic evaluations.

This direction is strategically stronger because prior authorization is not just a form problem. It is a multi-step operational problem: determine whether prior authorization is needed, identify payer documentation requirements, gather evidence, complete forms, package a submission, wait for payer response, respond to more-information requests, track denials, and preserve auditability. That workflow naturally fits a supervised agent system better than a single chatbot or a simple CRUD app.

The project should still avoid production overclaiming. CMS has pushed interoperability and prior authorization API requirements into the industry conversation, and the Da Vinci CRD, DTR, and PAS guides remain the right standards spine for this domain [R1][R2][R3][R4]. However, production readiness would still require real EHR launch behavior, payer endpoint discovery, live FHIR exchange, security controls, PHI-safe operations, formal testing, and implementation-specific integrations. The open-source target should be agentic and production-aware, not production-claiming.

Layer	Meaning	Expected state
Doctor Agent OS	Custom agent runtime, tool runtime, MCP server, workflow engine, policy layer, evaluation harness, and agent cockpit.	Build as the new technical heart of the project.
Open Prior Auth Agent Workbench	The first healthcare admin app built on Doctor Agent OS.	Keep prior authorization as the flagship scenario.
Standards Gateway	SMART/CRD/DTR/PAS/FHIR-shaped adapters and routes that map real standards to local synthetic tools.	Build enough to educate, test locally, and show maturity without claiming certification.
Production architecture path	Real EHR/payer integration, security, PHI handling, deployment hardening, and conformance path.	Document honestly; implement only after runtime and workflow foundations are stable.

Final goal: One-sentence source-of-truth goal:

Doctor Agent OS aims to be the best open-source, synthetic-data-only agentic reference system for healthcare administrative workflows, beginning with provider-side prior authorization. It demonstrates how a supervised agent team can discover requirements, gather documentation, map evidence, assemble a PAS-style packet, request human approval, submit to a mock payer, track payer determinations, and produce a full audit trace through standards-shaped healthcare adapters.

2. What changed from the previous strategy

The previous strategy separated the open-source reference workbench from the real production north star. That remains correct. The change is that the open-source target should now be more ambitious internally. Instead of only showing a standards-shaped prior authorization workflow, it should show how an agentic healthcare admin system should be built from first principles.

Area	Previous direction	Updated direction
Product identity	Open Prior Auth Workbench.	Doctor Agent OS with Open Prior Auth Agent Workbench as the flagship app.
Core demo	Synthetic CRD -> DTR -> PAS-style prior-auth flow.	Supervised multi-agent prior-auth run over the same CRD -> DTR -> PAS-style flow.
Runtime	Next.js app, TypeScript API, SQLite state, standards-shaped routes.	Custom agent runtime, custom tool runtime, durable workflow, MCP server, standards gateway, eval harness, and UI cockpit.
AI role	Optional future AI/document extraction layer.	Central but bounded: agents plan, call tools, detect gaps, draft outputs, and stop for human approvals.
Tooling	Local HTTP endpoints.	Internal tool registry and broker, exposed through HTTP APIs and MCP where safe.
Workflow	Application state transitions and mock payer updates.	Long-running agent runs with checkpoints, handoffs, approval gates, retries, and audit traces.
Standards posture	Standards-shaped and production-aware.	Same honest posture, now wrapped as tool contracts and standards gateway.
Evaluation	API tests and fixture behavior.	Golden agent traces, tool-call evals, safety evals, and scenario scorecards.

2.1 What should be preserved

Existing foundation	Keep?	How it maps into the new system
Synthetic prior-auth vertical slice	Yes	Becomes the flagship agent run.
Requirement discovery	Yes	Becomes RequirementDiscoveryAgent plus doctor.requirements.* tools.
Documentation workspace	Yes	Becomes DocumentationAgent plus questionnaire session tools.
Evidence attachments	Yes	Becomes EvidenceAgent and evidence mapping/gap tools.
PAS-style packet builder	Yes	Becomes PacketAssemblyAgent plus doctor.pas.* tools.
Operations queue	Yes	Becomes case queue plus agent-run status surface.
SQLite persistence	Yes	Good local durable source of truth for v0.x.
Conformance/gap matrix	Yes	Becomes stronger because every agent/tool claim can be mapped to standards support.

2.2 What should be de-emphasized

Area	Updated stance
Production conformance as near-term goal	Keep as long-term north star. Do not make it the immediate open-source definition of success.
Single workbench UI	Replace with agent cockpit plus workbench screens. The UI must explain what the agents did.
Local endpoints as the product	Local endpoints become tools behind a tool broker and standards gateway.
Generic chatbot framing	Avoid. This is a supervised healthcare admin agent system with typed tools and approval gates.
Full document intelligence now	Push later. Start with synthetic evidence mapping and traceability before extraction.

3. Revised goal, scope, and north-star separation

3.1 Final revised goal

Doctor Agent OS should be an open-source, synthetic-data-only agentic system for healthcare administrative workflows. Its first product application is Open Prior Auth Agent Workbench: a supervised prior authorization agent run that coordinates requirement discovery, DTR-style documentation, evidence mapping, packet assembly, human approval, mock PAS submission, payer determination tracking, and operational reporting.

The system should demonstrate production architecture thinking without making production claims. It should show how an implementation partner could evolve from fixture-backed tools to real EHR/payer integrations, but the default developer experience should remain safe, synthetic, reproducible, and non-PHI.

3.2 North-star separation

Layer	What it means	What we build now	What remains production path
Agentic OSS target	A complete local system that runs supervised agents over synthetic prior-auth cases.	Custom runtime, tool broker, MCP server, workflow engine, agent cockpit, evals, standards-shaped tools.	Real PHI, real payer/EHR integrations, enterprise authz, production operations.
Standards-shaped interface harness	Routes and payloads that mirror SMART, CRD/CDS Hooks, DTR, PAS, and FHIR concepts.	Fixture-backed standards gateway with conformance/gap matrix and OperationOutcome-style errors.	Formal conformance testing, vendor-specific integration, endpoint discovery, live transport.
Production implementation north star	What a real deployment would require.	Document architecture, interfaces, security model, migration path.	Build only after foundations, partnerships, and compliance posture are ready.

3.3 What we are aiming to build now

Build target	Concrete deliverable	Why it matters
Agentic vertical slice	MRI prior-auth case run with orchestrator + specialists, typed tool calls, evidence gaps, human approval, packet preview, mock payer result.	Shows agentic workflow depth without real PHI or real payer risk.
Custom runtime	AgentSpec, AgentRunner, RunState, TaskPlan, Handoff, ApprovalGate, TraceRecorder, PolicyEngine.	Proves the project is not just wiring an LLM to endpoints.
Tool runtime	Tool registry, JSON schema validation, permission checks, risk levels, trace records, typed results.	Makes agent actions inspectable, safe, and testable.
MCP server	Local stdio MCP server exposing safe tools, resources, and prompts.	Shows interoperability with agent hosts while keeping internal tool runtime canonical.
Standards Gateway	SMART-shaped metadata, CDS Hooks discovery, CRD hook services, DTR questionnaire package, PAS Claim/\$submit mock.	Preserves healthcare standards credibility.
Agent cockpit	Plan, handoffs, tool calls, evidence map, approval panel, packet preview, audit trace, eval dashboard.	Lets users understand and trust what happened.
Evaluation harness	Golden traces, scenario scorecards, tool-call evals, safety evals, prompt-injection fixtures.	Turns agent behavior into reproducible engineering evidence.

3.4 Claim language

Do say	Do not say
Open-source agentic healthcare admin reference system.	Production healthcare automation platform.
Synthetic-data local agent run.	Ready for real PHI.
Standards-shaped CRD/DTR/PAS workflow.	Certified Da Vinci implementation.
Mock PAS transport and mock payer determinations.	Live payer submission.
MCP-enabled tool boundary.	Production MCP security posture.
Human-reviewed administrative workflow agent.	Autonomous clinical decision-maker.
Production-readiness gap map.	Production-ready claim.

4. Product identity and recommendation

Working project identity: Doctor Agent OS.

First product application: Open Prior Auth Agent Workbench.

Core pitch: a supervised agentic system that helps healthcare teams turn fragmented administrative workflows into auditable, standards-shaped, human-reviewed agent runs.

Naming layer	Recommended name	Purpose
Overall project	Doctor Agent OS	The broad system identity that can grow beyond prior auth.
Prior-auth app	Open Prior Auth Agent Workbench	The first flagship workflow/product surface.
Agent runtime	Doctor Runtime	Custom orchestration engine for agent runs, handoffs, state, and approvals.
Tool runtime	Doctor ToolNet	Tool registry, validation, execution, permissions, audit.
Healthcare standards layer	Doctor Standards Gateway	SMART/CRD/DTR/PAS/FHIR-shaped adapter layer.
MCP boundary	Doctor MCP	External agent interoperability through resources, prompts, and tools.
Evaluation system	Doctor Evals	Golden traces, scorecards, safety checks, scenario replays.

4.1 Product recommendation

Decision area	Recommendation	Reason
Primary wedge	Provider-side prior authorization.	High operational pain, standards spine, strong demo narrative, feasible synthetic OSS target.
Product type	Agentic reference system plus workbench.	Combines product usefulness with architecture depth.
AI posture	Supervised agents with typed tools and human approval gates.	Healthcare admin needs accountability, not unconstrained autonomy.
Standards posture	Standards-shaped and test-path aware.	Demonstrates healthcare literacy while avoiding false conformance claims.
Data posture	Synthetic-first, fixture-backed, deterministic.	Safe for OSS demos and reproducible evals.
Integration posture	Adapters before live integrations.	Keeps production path open without blocking the MVP.

5. Agentic PRD

5.1 Product vision

Reduce manual healthcare administrative burden by turning payer requirements, documentation tasks, evidence gathering, submission packaging, review checkpoints, and payer status updates into supervised, auditable agent runs. The system should run locally with synthetic data, expose standards-shaped healthcare boundaries, and later evolve toward real provider/payer integration through replaceable adapters.

5.2 Product principles

- Agentic, not magical: agents coordinate work, but tools own facts and state changes.
- Human-in-the-loop by default: any write, packet submission, or case-changing step can pause for approval.
- FHIR-first, not FHIR-only: clinical and payer context should stay close to FHIR resources while app workflow state remains app-owned.
- MCP-exposed, not MCP-owned: MCP is an external protocol boundary, while Doctor ToolNet remains the internal tool source of truth.
- Synthetic-first: every scenario must run without PHI, contracts, payer credentials, or EHR access.
- Trace everything: each agent decision, tool call, evidence mapping, approval, and status transition should be inspectable.
- Scope honesty: the project should clearly label fixture-backed, standards-shaped, tested, partial, and unsupported capabilities.

5.3 Jobs to be done

User / actor	Job to be done	Agentic system response
Ordering clinician	When a service is ordered, tell me if prior authorization is likely needed and what happens next.	CaseIntakeAgent and RequirementDiscoveryAgent evaluate synthetic context and summarize requirements.
Authorization specialist	When prior auth is required, give me the right documentation workspace and show missing data clearly.	DocumentationAgent retrieves DTR-style package, prefills answers, validates missing fields, and logs provenance.
Evidence reviewer	When evidence is attached, tell me which requirement it supports and whether anything is still missing.	EvidenceAgent maps evidence to requirements and produces a gap report.
Supervisor/reviewer	Before submission, let me inspect the packet and approve or reject the action.	PacketAssemblyAgent builds preview and stops at ApprovalGate.
Operations lead	Show me queue, aging, pending cases, more-info loops, denials, and throughput.	OpsReportingAgent summarizes operational state from case and agent traces.
Developer/implementer	Let me understand tools, payloads, standards mapping, gaps, and how to extend the system.	MCP/HTTP catalogs, conformance matrix, fixtures, docs, and eval traces are visible.

5.4 User stories for v0.1

1. As a developer, I can clone, install, seed, run, and open a synthetic prior-auth agent demo locally.
2. As a clinician persona, I can open a synthetic order and see whether prior authorization is likely required.
3. As an authorization specialist, I can start an agent run and watch the orchestrator create a plan.
4. As a reviewer, I can see every tool call the agents made and inspect the inputs and outputs.
5. As an authorization specialist, I can see DTR-style questionnaire fields, prefilled values, and missing-data reasons.
6. As an evidence reviewer, I can see evidence mapped to specific payer requirements.
7. As a supervisor, I can approve or reject mock packet submission.
8. As an operations lead, I can see case state, effective status, aging, more-info, denials, approvals, and payer determinations.

9. As an implementer, I can query Doctor MCP for cases, evidence manifests, standards-shaped artifacts, and safe tools.
10. As a maintainer, I can run golden trace evals and detect behavior regressions in agent planning and tool use.

5.5 Scope

Release / layer	In scope	Out of scope
v0.1 Agentic MVP	Custom runtime, tool registry, multi-agent MRI prior-auth run, agent cockpit, human approval, mock payer, golden trace.	Real PHI, live payer/EHR, certification, autonomous submission, clinical decision-making.
v0.2 Standards Gateway	SMART-shaped metadata, CDS Hooks discovery, CRD services, DTR questionnaire package, PAS Claim/\$submit mock, conformance matrix.	Claiming formal Da Vinci conformance.
v0.3 Reusable workflow proof	Second payer, second service line, shared rules, differentiated questionnaires, queue filters, alternative payer outcomes.	General RCM/claims platform.
v0.4 Durable agent workflow	SQLite/Postgres run store, pause/resume, retry semantics, approval persistence, status inquiry workflow, Temporal path.	Full enterprise deployment.
Future production path	FHIR data-plane adapter, auth/security/audit, document intelligence, sandbox integration pilot.	Uncontrolled autonomous clinical/admin actions.

5.6 Non-goals

- Do not automate final payer adjudication.
- Do not replace EHR clinical source-of-truth systems.
- Do not market the system as a clinical AI decision-maker.
- Do not claim production readiness, certification, PHI readiness, or live payer connectivity.
- Do not make a third-party agent framework the canonical runtime if the project goal is to build from scratch.
- Do not expose write/submit tools through MCP without approval gates and risk controls.
- Do not broaden the first wedge into full revenue cycle management before the prior-auth workflow is strong.

5.7 Success metrics

Metric category	Candidate metric
Demo completion	Percent of seeded scenarios that complete from intake to payer determination.
Agent legibility	Percent of steps with visible plan item, responsible agent, tool call, output, and audit event.
Documentation quality	Percent of questionnaire fields prefilled from synthetic FHIR context with provenance.
Gap detection	Percent of cases with explicit missing-data reasons and mapped evidence requirements.
Approval safety	Percent of write/submit operations that pass through ApprovalGate in reviewed-live mode.
Tool reliability	Tool-call success/error/blocked/approval-needed rates.
Determinism	Golden trace match rate across fixture-backed runs.
Reuse	Number of payer/service-line combinations using the same engine and tool contracts.
Developer success	Clone -> install -> seed -> run -> inspect trace success rate.

6. Users, personas, and agent-facing jobs to be done

6.1 Human personas

Persona	Needs	Primary screens
Clinician/orderer	Quick administrative clarity: is prior auth needed, what is missing, and who owns next step?	Order launch, requirement result, case summary.
Authorization specialist	Queue, forms, missing-data explanations, evidence attachment, status tracking.	Case queue, documentation workspace, evidence board.
Reviewer/supervisor	Packet preview, validation, evidence map, approval/rejection controls, audit trace.	Packet preview, approval panel, audit trace.

Persona	Needs	Primary screens
Ops lead	Aging, pending volume, denials, more-info loops, turnaround time, bottlenecks.	Operations dashboard, metrics, case queue.
Developer/implementer	Schemas, fixtures, local run, MCP catalog, standards mapping, tests, extension points.	MCP inspector, docs, eval dashboard.

6.2 Agent personas

Agent	Job	Allowed tool families
PriorAuthOrchestratorAgent	Plan the case run, coordinate specialists, enforce sequence, summarize progress.	workflow, case, audit, specialist handoff.
CaseIntakeAgent	Convert order/context into a case and summarize administrative context.	case, fhir, audit.
RequirementDiscoveryAgent	Evaluate payer/service-line rules and explain prior-auth/documentation requirements.	requirements, crd, coverage.
DocumentationAgent	Retrieve, prefill, validate, and save DTR-style questionnaire sessions.	dtr, questionnaire, fhir.
EvidenceAgent	List evidence, map it to requirements, identify gaps, preserve provenance.	evidence, document reference, audit.
PacketAssemblyAgent	Build deterministic PAS-style packet and request approval before submission.	pas, workflow, audit.
PayerFollowupAgent	Handle pending, more-info, approved, denied, cancelled, and next actions.	payer, status, case.
ComplianceBoundaryAgent	Check PHI, standards claims, approval gates, medical-advice boundaries, and unsafe tool calls.	policy, audit, eval.
OpsReportingAgent	Explain queues, aging, bottlenecks, denial/more-info metrics.	ops, audit, metrics.

7. Agent system design

Use a manager-plus-specialists pattern. The orchestrator owns the case-level plan and delegates bounded tasks to specialist agents. Specialist agents should not own arbitrary write access; they should call typed tools through Doctor ToolNet. The ComplianceBoundaryAgent can run before risky actions and after drafted outputs.

```
PriorAuthOrchestratorAgent
  |-- CaseIntakeAgent
  |-- RequirementDiscoveryAgent
  |-- DocumentationAgent
  |-- EvidenceAgent
  |-- PacketAssemblyAgent
  |-- PayerFollowupAgent
  |-- DenialAppealAgent
  |-- ComplianceBoundaryAgent
  `-- OpsReportingAgent
```

7.1 Core agent outputs

Output	Purpose	Example fields
TaskPlan	Structured plan for a case run.	goal, steps, owner_agent, required_tools, approval_points, blockers.
AgentTaskResult	Result of a specialist task.	status, summary, evidence_refs, tool_call_ids, next_actions.
CaseRunSummary	Human-readable status summary.	case_id, current_phase, blocked, missing_items, approval_required.
SafetyFinding	Policy or boundary finding.	risk_type, severity, blocked_action, rationale, remediation.
AuditExplanation	Trace-friendly explanation.	what_happened, why, data_used, tools_called, human_actions.

7.2 Agent run states

created

- > planning
- > executing
- > waiting_for_tool
- > waiting_for_human
- > resumed
- > completed | blocked | failed | cancelled

7.3 Case lifecycle with agent overlay

intake_created

- > requirements_discovered
- > documentation_in_progress
- > evidence_review_needed
- > review_ready
- > packet_ready
- > human_approval_required
- > submitted
- > pended | more_info_needed
- > approved | denied | cancelled

7.4 Why not one giant agent

Problem with one giant agent	Specialist-agent answer
Too much instruction complexity.	Each agent has a narrow role, schema, tool allowlist, and escalation policy.
Hard to debug failures.	Run traces show which specialist made which decision and tool call.
Unsafe broad tool access.	Tool allowlists and risk levels differ by agent.
Poor evaluation granularity.	Each specialist can have focused golden traces and scoring criteria.
Bad product legibility.	The cockpit can explain handoffs and responsibilities to users.

8. Doctor Runtime - custom agent runtime

Doctor Runtime is the from-scratch orchestration layer. It should borrow useful concepts from mature agent SDKs - agents with instructions, tools, handoffs, guardrails, structured outputs, sessions, and tracing - but the project should own its own healthcare-specific semantics and safety boundaries [R11][R12].

8.1 Core package

packages/doctor-runtime/
src/agent-spec.ts
src/agent-runner.ts
src/run-state.ts
src/task-plan.ts
src/handoff.ts
src/approval-gate.ts
src/policy-hooks.ts
src/trace-recorder.ts
src/output-validator.ts
src/replay-runner.ts

8.2 Runtime components

Component	Responsibility
AgentSpec	Defines name, role, instructions, allowed tools, output schema, handoff targets, escalation policy.
AgentRunner	Executes agent turns, dispatches tool calls, validates outputs, records traces.

Component	Responsibility
RunState	Durable state for an agent run, including current phase, tasks, blockers, approvals.
TaskPlan	Structured plan that can be reviewed, modified, and compared in evals.
Handoff	Typed transfer from one agent to another with task context and expected output schema.
ApprovalGate	Pauses execution before risky operations such as write, status update, or submission.
PolicyEngine	Checks whether the agent, tool, input, and case mode permit the action.
TraceRecorder	Captures model outputs, tool calls, approvals, state changes, policy checks, and errors.
ReplayRunner	Replays fixture-backed runs deterministically for tests and demos.

8.3 Execution modes

Mode	Behavior	Use
replay	No live model or external tools; use golden fixture outputs.	CI, demos, deterministic evals.
live-local	Agents may call local synthetic tools and local/mock models.	Developer experimentation.
reviewed-live	Agents may propose case-changing actions, but writes/submissions pause for human approval.	Primary interactive demo mode.
production-pilot	Future mode with real integrations, strict auth, audit, redaction, observability, and approvals.	Out of v0.1 scope.

8.4 Minimal TypeScript contracts

```
type AgentSpec = {
  name: string;
  role: string;
  instructions: string;
  allowedTools: string[];
  outputSchema: JsonSchema;
  handoffTargets?: string[];
  escalationPolicy?: EscalationPolicy;
};

type AgentRun = {
  id: string;
  caseId: string;
  mode: 'replay' | 'live-local' | 'reviewed-live';
  status: 'created' | 'planning' | 'executing' | 'waiting_for_human' | 'completed' | 'blocked' | 'failed';
  plan?: TaskPlan;
  tasks: AgentTask[];
  traceIds: string[];
  createdAt: string;
  updatedAt: string;
};
```

9. Doctor ToolNet - custom tool runtime

Doctor ToolNet is the internal tool system. It is the canonical source of available actions. HTTP routes and MCP tools should adapt to ToolNet rather than duplicating logic. This prevents standards-shaped routes, UI buttons, and agents from each implementing separate behavior.

9.1 Core package

```
packages/doctor-toolnet/  
  src/tool-definition.ts  
  src/tool-registry.ts  
  src/tool-broker.ts  
  src/schema-validator.ts  
  src/policy-checker.ts  
  src/approval-router.ts  
  src/tool-call-recorder.ts  
  src/errors.ts  
  src/catalog.ts
```

9.2 Tool definition

```
type DoctorTool = {  
  name: string;  
  description: string;  
  inputSchema: JsonSchema;  
  outputSchema: JsonSchema;  
  category: 'case' | 'fhir' | 'crd' | 'dtr' | 'pas' | 'evidence' | 'payer' | 'workflow' | 'eval' | 'system';  
  riskLevel: 'read' | 'draft' | 'write' | 'submit' | 'external';  
  requiresApproval: boolean;  
  allowedAgents?: string[];  
  handler: ToolHandler;  
};
```

9.3 Tool-call lifecycle

```
agent proposes tool call  
  -> input schema validation  
  -> agent/tool permission check  
  -> case-mode policy check  
  -> approval check if required  
  -> tool execution  
  -> output schema validation  
  -> trace logging  
  -> normalized result returned to agent
```

9.4 Tool risk model

Risk level	Examples	Default approval
read	Get case, list evidence, inspect questionnaire, read audit trace.	No approval in local modes.
draft	Draft packet, draft appeal text, summarize denial, preview questionnaire.	No approval, but trace required.
write	Save response, attach evidence, update case status.	Approval in reviewed-live mode.
submit	Submit mock PAS packet or send response to payer connector.	Always approval.
external	Any real EHR, payer, email, Slack, or web action.	Out of v0.1; always approval if later enabled.

9.5 Tool-call record

```
type ToolCallRecord = {  
  id: string;  
  runId: string;  
  caseId: string;  
  agentName: string;
```

```
toolName: string;
riskLevel: ToolRiskLevel;
input: unknown;
output?: unknown;
status: 'success' | 'error' | 'blocked' | 'needs_approval';
startedAt: string;
completedAt?: string;
approvalId?: string;
error?: OperationOutcomeLike;
};
```

10. Doctor MCP - external agent interoperability boundary

MCP should be implemented as an adapter over Doctor ToolNet. MCP is useful because it gives external AI hosts a consistent way to discover tools, resources, and prompts. MCP should not be the internal source of truth. The internal tool registry remains Doctor ToolNet; MCP exposes selected capabilities safely [R8][R9][R10].

10.1 Core package

```
apps/mcp-server/
packages/doctor-mcp/
  src/server.ts
  src/toolnet-adapter.ts
  src/resources.ts
  src/prompts.ts
  src/transport/stdio.ts
  src/transport/http.ts
  src/auth-policy.ts
```

10.2 MCP mapping

MCP primitive	Doctor mapping	Examples
Resources	Read-only case, evidence, standards, and audit artifacts.	case summaries, FHIR bundles, evidence manifest, packet preview, golden trace.
Prompts	Reusable workflow prompts that describe a bounded task.	run prior-auth case, explain denial, find missing evidence, prepare more-info response.
Tools	Safe ToolNet tools exposed through MCP.	doctor.case.get, doctor.evidence.list, doctor.audit.get_trace.
Elicitation	Future human data request pattern.	Ask staff for missing clinical admin details.
Sampling	Future optional model delegation.	Out of MVP unless needed for host-specific flows.

10.3 Initial MCP tools

```
doctor.case.list
doctor.case.get
doctor.case.create_from_order
doctor.requirements.evaluate
doctor.dtr.get_questionnaire_package
doctor.dtr.save_response
doctor.evidence.list
doctor.evidence.attach
doctor.evidence.check_gaps
doctor.pas.build_packet
doctor.pas.submit_mock
doctor.payer.record_status
doctor.workflow.request_human_approval
doctor.audit.get_trace
doctor.eval.run_scenario
```

10.4 MCP safety rule

Expose read-only tools first. For mutation, packet submission, external calls, file writes, and anything that could change case state, Doctor MCP should require policy checks and approval gates. The protocol boundary should assume that model outputs and external content can be malicious or confused. OWASP specifically highlights prompt injection, insecure output handling, supply-chain issues, sensitive information disclosure, and excessive agency as LLM application risks [R14][R15].

11. Standards gateway and conformance honesty

The project should preserve the standards posture from the previous strategy. It should mirror the healthcare workflow sequence - SMART launch/context, CRD requirement discovery, DTR documentation package, PAS packet submission - while clearly labeling fixture-backed and non-certified behavior.

11.1 Standards to mirror

Standard / guide	Production role	Doctor OSS target
SMART App Launch	EHR launches app with context and OAuth-based authorization.	SMART-shaped discovery and local launch adapter. No production EHR launch claim.
CDS Hooks / CRD	Coverage requirement discovery during clinical workflow.	CDS Hooks-style discovery and CRD-like services backed by synthetic rule packs.
DTR questionnaire-package	Payer supplies documentation templates and dependencies.	FHIR Operation-like package returning synthetic Questionnaire, QuestionnaireResponse, ValueSet, Library, metadata.
PAS Claim/\$submit	Provider submits prior-auth request and receives payer response.	Deterministic local packet submission through mock payer transport.
FHIR R4 resources	Patient, Coverage, ServiceRequest, Questionnaire, Claim, ClaimResponse, DocumentReference, Task, AuditEvent, etc.	FHIR-shaped fixtures and adapter boundary; not real production FHIR data plane.
Inferno / Da Vinci test kits	Conformance test path for implementations.	Track test path and gaps; no certification claim until actual conformance is achieved.

11.2 Conformance matrix

Area	OSS support target	Production claim allowed?	Honest language
SMART	Fixture/local metadata and launch-context adapter.	No	SMART-shaped local harness; not production EHR launch.
CRD	CDS Hooks-shaped service discovery and synthetic requirement responses.	No	CRD-inspired requirement discovery with fixture-backed payer rules.
DTR	Questionnaire-package-shaped operation with synthetic package bundle.	No	DTR-shaped package flow; not certified DTR conformance.
PAS	Claim/\$submit-shaped route, packet builder, mock payer response.	No	PAS-style packet assembly and mock transport.
FHIR data	FHIR-shaped fixtures and repository adapter.	No	FHIR-first model; fixture-backed until real FHIR adapter is implemented.
MCP	Local server exposing safe Doctor tools/resources/prompts.	No production security claim	MCP-enabled local agent boundary; write tools approval-gated.
Agent runtime	Custom runtime with replay/live-local/reviewed-live modes.	No autonomous production claim	Supervised agent workflow with audit and approvals.
Security	Synthetic data only, PHI-safe logging principles, policy docs.	No	Not PHI-ready; security architecture documented.

11.3 Standards-shaped routes

Route	Purpose	Internal ToolNet mapping
GET /fhir/.well-known/smart-configuration	SMART-shaped discovery metadata.	doctor.smart.get_configuration
GET /cds-services	CDS Hooks service discovery.	doctor.crd.discover_services
POST /cds-services/open-prior-auth-order-sign	CRD-style order-sign service.	doctor.requirements.evaluate
POST /cds-services/open-prior-auth-appointment-book	CRD-style appointment-book service.	doctor.requirements.evaluate
POST /fhir/Questionnaire/\$questionnaire-package	DTR-style questionnaire package operation.	doctor.dtr.get_questionnaire_package
POST /fhir/Claim/\$submit	PAS-style claim submit operation.	doctor.pas.submit_mock

12. End-to-end agentic workflow design

The workflow should be shown as a supervised agent run over the CRD -> DTR -> PAS-style spine. The UI should allow a human to understand both the business process and the tool trace behind it.

12.1 Workflow 1: case intake and planning

11. User opens a synthetic patient/order context or starts a seeded scenario.
12. PriorAuthOrchestratorAgent creates an initial TaskPlan.
13. CaseIntakeAgent reads synthetic FHIR-shaped context and creates/updates the case.
14. ComplianceBoundaryAgent verifies synthetic/demo mode and prevents real PHI claims.
15. The cockpit shows the plan, current phase, next tool calls, and expected approval gates.

12.2 Workflow 2: requirement discovery

16. RequirementDiscoveryAgent calls doctor.requirements.evaluate or CRD-shaped service adapter.
17. The requirement engine evaluates payer, service line, order context, coverage, and rule pack.
18. Tool output returns requires_prior_auth, documentation_needed, missing_baseline_data, questionnaire hints, and explanation.
19. The system records a requirement run and audit event.
20. The UI shows both human-friendly requirement results and the standards-shaped CRD/CDS response artifact.

12.3 Workflow 3: documentation workspace

21. DocumentationAgent calls doctor.dtr.get_questionnaire_package.
22. The questionnaire service returns Questionnaire, draft QuestionnaireResponse, ValueSets, Libraries, rule metadata, and prefill provenance.
23. DocumentationAgent calls prefill and validation tools.
24. The specialist completes missing fields or accepts agent-suggested draft values.
25. Material changes produce questionnaire session records and audit events.

12.4 Workflow 4: evidence mapping and gap detection

26. EvidenceAgent lists existing evidence and fixtures.
27. EvidenceAgent maps evidence items to payer requirements.
28. The system identifies missing items, weak evidence, or conflicting evidence.
29. The cockpit shows a requirement-to-evidence board.
30. A case can be blocked until missing evidence is attached or a reviewer accepts a warning.

12.5 Workflow 5: packet assembly and human approval

31. PacketAssemblyAgent calls doctor.pas.build_packet.
32. The system builds a deterministic PAS-style packet with evidence manifest and idempotency key.
33. Packet preview appears in the UI.
34. ApprovalGate pauses before doctor.pas.submit_mock.
35. Reviewer approves, rejects, or sends back to documentation/evidence workspace.

12.6 Workflow 6: payer response, more-info, denial, and closeout

36. Mock payer returns pended, approved, denied, cancelled, or more-info-needed response.
37. PayerFollowupAgent summarizes payer update and next action.
38. More-info-needed routes the case back to documentation/evidence without losing packet history.
39. Denial reasons are parsed into structured explanation and optional appeal draft task.
40. Final payer determination closes the payer response cycle while preserving full audit trace.

13. Agent cockpit UI and demo narrative

The UI must make agent behavior visible. The core question for the user should not be "what did the chatbot say?" It should be "what is the case status, what did the agents do, what evidence did they use, what tool calls changed state, and what needs human approval?"

13.1 Main screens

Screen	Purpose
Case Queue	Cases, status, effective status, aging, owner, next action, payer state.
Case Detail	Patient/order/coverage context, requirements, evidence, documentation, packet, timeline.

Screen	Purpose
Agent Run Timeline	Plan, tasks, handoffs, tool calls, approvals, policy checks, failures.
Evidence Board	Requirement-to-evidence mapping, gaps, weak evidence, accepted/rejected evidence.
Questionnaire Workspace	DTR-style questionnaire rendering, prefill, validation, missing-data explanations.
Packet Preview	PAS-style packet, attachments manifest, receipt preview, approval controls.
Payer Updates	Pended, more-info, approved, denied, cancelled, reasons, next action.
Audit Trace	Full explanation of what happened, by actor/agent/tool/time.
Eval Dashboard	Scenario pass/fail, golden trace diffs, safety evals, regression checks.
MCP Inspector	Resources, prompts, tools, schemas, allowed/blocked tool states.

13.2 Agent cockpit layout

```
+-----+
| Case: MRI Lumbar Spine Prior Auth                                     |
| Status: Evidence Review Needed | Agent Run: waiting_for_human      |
+-----+-----+
| Agent Plan          | Case Context          |
| 1. Evaluate rules   ok | Patient: Synthetic    |
| 2. Get questionnaire ok | Order: MRI lumbar spine |
| 3. Map evidence     ! | Payer: Synthetic Blue  |
| 4. Build packet     ... | Service line: Imaging  |
+-----+-----+
| Tool Calls          | Evidence Gaps          |
| requirements.evaluate ok | Missing: recent neurologic exam |
| dtr.get_package     ok | Matched: PT note, medication hx |
| evidence.check_gaps  ! |                               |
+-----+-----+
| Human approval: attach missing evidence or proceed with warning? |
+-----+-----+
```

13.3 Demo story script

- 41. Open the Case Queue and pick the MRI lumbar spine synthetic case.
 - 42. Start an agent run. The orchestrator creates a visible plan.
 - 43. RequirementDiscoveryAgent calls the requirement tool and shows that prior authorization is required.
 - 44. DocumentationAgent retrieves the DTR-style package and prefills known answers.
 - 45. EvidenceAgent maps PT notes and medication history, then flags a missing neurologic exam.
 - 46. The human attaches synthetic evidence or accepts a warning.
 - 47. PacketAssemblyAgent builds a PAS-style packet and pauses for approval.
 - 48. Reviewer opens Packet Preview, inspects evidence and questionnaire response, and approves mock submission.
 - 49. Mock payer returns pended, approved, denied, or more-info-needed.
 - 50. The final Audit Trace shows the full story: agents, tools, evidence, approvals, and payer determination.
- Analogy: the system is not a robot doctor. It is an air traffic controller for healthcare paperwork. Agents coordinate the moving parts, but every risky action is routed through instruments, rules, logs, and human approval.

14. Reference architecture and monorepo structure

14.1 Architecture principles

- The runtime owns the run: agent state, handoffs, approvals, and traces belong to Doctor Runtime.
- The tool broker owns actions: agents do not mutate state directly.
- The workflow layer owns durable progress: long-running case state, timers, retries, and waits are explicit.
- The standards gateway owns healthcare protocol shape: SMART/CRD/DTR/PAS routes adapt to internal tools.
- The UI owns legibility: the cockpit shows plan, evidence, tools, approval gates, and trace.
- The eval layer owns trust: golden traces and safety tests define acceptable agent behavior.

14.2 Logical architecture

```
Web app / Agent Cockpit
-> Product API Gateway
-> Doctor Runtime
    -> PriorAuthOrchestratorAgent
    -> Specialist agents
-> Doctor ToolNet
    -> Case tools
    -> FHIR/context tools
    -> Requirements/CRD tools
    -> DTR tools
    -> Evidence tools
    -> PAS tools
    -> Payer/status tools
    -> Workflow/audit/eval tools
-> Doctor Standards Gateway
    -> SMART-shaped routes
    -> CDS Hooks / CRD-shaped routes
    -> DTR questionnaire-package route
    -> PAS Claim/$submit route
-> Doctor MCP Server
    -> Resources
    -> Prompts
    -> Safe tools
-> SQLite/Postgres run store + fixture data
-> Doctor Evals + observability
```

14.3 Proposed monorepo

```
doctor-agent-os/
apps/
  web/          # Next.js agent cockpit + prior-auth workbench
  api/          # HTTP product API and standards routes
  mcp-server/   # Doctor MCP stdio/http server
  worker/       # Agent/workflow execution worker
  demo-console/ # CLI scenario runner
packages/
  doctor-runtime/ # Agent runner, state, handoffs, approvals, traces
  doctor-toolnet/ # Tool registry, broker, schema validation, policies
  doctor-mcp/     # MCP adapter over ToolNet
  doctor-workflow/ # Durable workflow/checkpoint/retry semantics
  doctor-standards/ # SMART/CRD/DTR/PAS-shaped contracts
  doctor-fhir/    # FHIR-shaped types and bundle utilities
  doctor-cases/   # Case lifecycle and repositories
  doctor-evidence/ # Evidence metadata, mapping, gap analysis
  doctor-evals/   # Scenarios, golden traces, scorecards
  doctor-policy/  # PHI, approval, standards-claim, safety policies
  shared-types/
services/
  mock-ehr/
  mock-payer/
  mock-identity/
data/
  fixtures/
  scenarios/
  golden-traces/
```



```
docs/
  product/
  architecture/
  standards/
  evals/
  security/
tests/
  unit/
  contract/
  integration/
  e2e/
  evals/
infra/
  compose/
  sqlite/
  otel/
```

15. Core data model

15.1 Domain entities

Case
OrderContext
PatientContext
CoverageContext
RequirementResult
DocumentationWorkspace
QuestionnaireSession
EvidenceItem
EvidenceMapping
SubmissionPacket
SubmissionReceipt
PayerUpdate
PayerDetermination
MoreInfoRequest
AgentRun
AgentTask
ToolCallRecord
ApprovalRequest
AuditEvent
EvalScenario
GoldenTrace

15.2 App-owned operational tables

Table	Key fields	Purpose
cases	id, patient_id, coverage_id, request_resource_type, request_resource_id, payer_id, service_line, status, effective_status, owner, created_at, updated_at.	Case queue and current operational state.
agent_runs	id, case_id, mode, status, plan_json, current_agent, created_at, updated_at, completed_at.	Agent-run state and lifecycle.
agent_tasks	id, run_id, case_id, agent_name, task_type, status, input_json, output_json, blocker_json.	Specialist task tracking.
tool_call_records	id, run_id, case_id, agent_name, tool_name, risk_level, input_json, output_json, status, error_json, timestamps.	Tool audit and debugging.
approval_requests	id, run_id, case_id, requested_action, risk_level, status, requested_by, decided_by, timestamps.	Human approval gates.
requirement_runs	case_id, payer_id, rule_pack_version, result_json, requires_prior_auth, missing_data_json.	Requirement discovery trace.
questionnaire_sessions	case_id, questionnaire_canonical, response_json,	Editable documentation workspace.

Table	Key fields	Purpose
	completion_percent, validation_errors, prefill_provenance_json.	
evidence_items	case_id, document_reference_json, content_pointer, checksum, byte_size, mime_type, uploaded_by, uploaded_at.	Evidence metadata and provenance.
evidence_mappings	case_id, evidence_item_id, requirement_id, strength, rationale, accepted_by, accepted_at.	Requirement-to-evidence mapping.
submission_packets	case_id, packet_version, claim_json, attachments_manifest_json, idempotency_key, transport_mode, tracking_id.	Deterministic packet history.
payer_updates	case_id, payer_status, determination, denial_reason_json, more_info_request_json, occurred_at, source.	Payer response model.
audit_events	actor_type, actor_id, action, resource_type, resource_id, before_json, after_json, timestamp, correlation_id.	Compliance posture and debugging.

15.3 FHIR-shaped resources

Resource family	Resources / artifacts
Clinical/context	Patient, Coverage, Encounter, Practitioner, PractitionerRole, Organization, ServiceRequest, MedicationRequest, DeviceRequest.
Questionnaire/documentation	Questionnaire, QuestionnaireResponse, ValueSet, Library, Parameters, Provenance.
Evidence/attachments	DocumentReference, Binary, Attachment manifest, Bundle-like evidence package.
Submission/payer response	Claim or PAS-style packet analog, ClaimResponse or payer determination equivalent, OperationOutcome.
Workflow/audit	Task-like artifact, AuditEvent, Provenance, local status events.

16. API, tool, and MCP catalog

16.1 Tool families

Family	Representative tools
Case	doctor.case.list, doctor.case.get, doctor.case.create_from_order, doctor.case.update_status, doctor.case.add_note.
FHIR/context	doctor.fhir.get_patient, doctor.fhir.get_coverage, doctor.fhir.get_service_request, doctor.fhir.get_bundle, doctor.fhir.validate_bundle.
Requirements/CRD	doctor.requirements.evaluate, doctor.requirements.explain, doctor.requirements.list_missing, doctor.crd.discover_services, doctor.crd.invoke_service.
DTR	doctor.dtr.get_questionnaire_package, doctor.dtr.prefill_response, doctor.dtr.validate_response, doctor.dtr.save_response.
Evidence	doctor.evidence.list, doctor.evidence.get, doctor.evidence.attach_fixture, doctor.evidence.map_to_requirement, doctor.evidence.check_gaps.
PAS	doctor.pas.build_packet, doctor.pas.validate_packet, doctor.pas.preview_packet, doctor.pas.submit_mock, doctor.pas.get_receipt.
Payer/status	doctor.payer.record_status, doctor.payer.request_more_info, doctor.payer.record_determination, doctor.payer.explain_denial.
Workflow	doctor.workflow.start_run, doctor.workflow.pause, doctor.workflow.resume, doctor.workflow.request_approval, doctor.workflow.approve, doctor.workflow.reject.
Audit/eval	doctor.audit.get_trace, doctor.audit.export_case, doctor.eval.run_scenario, doctor.eval.compare_golden_trace, doctor.eval.score_run.

16.2 Product-facing API shape

Endpoint	Purpose
POST /agent-runs	Start an agent run for a case/scenario.
GET /agent-runs/:id	Read run state, plan, status, and current blocker.
GET /agent-runs/:id/timeline	Read agent tasks, handoffs, tool calls, approvals, policy checks.

Endpoint	Purpose
POST /agent-runs/:id/resume	Resume a paused run after approval or added evidence.
POST /approvals/:id/approve	Approve a case-changing action.
POST /approvals/:id/reject	Reject a proposed action.
GET /cases/:id/evidence-map	Requirement-to-evidence map and gaps.
GET /cases/:id/packet-preview	Human review packet preview.
POST /evals/scenarios/:id/run	Run deterministic scenario.

16.3 MCP resources and prompts

MCP item	Examples
Resources	doctor://cases/{id}, doctor://cases/{id}/audit, doctor://cases/{id}/evidence, doctor://packets/{id}, doctor://evals/golden-traces/{id}.
Prompts	run_prior_auth_case, explain_payer_denial, find_missing_evidence, prepare_more_info_response, summarize_agent_trace.
Safe tools first	case.get, evidence.list, audit.get_trace, eval.run_scenario.
Approval-gated tools	dtr.save_response, evidence.attach, pas.submit_mock, payer.record_status.

17. Evaluation harness and golden traces

Agent systems need behavior evaluation, not just endpoint tests. Doctor Evals should make expected agent behavior explicit and reproducible. Every important scenario should have a golden trace that covers plan, tool calls, outputs, approvals, policy checks, and final case state.

17.1 Evaluation artifacts

Artifact	Purpose
EvalScenario	A seeded patient/order/payer/evidence setup plus expected behavior.
GoldenTrace	Expected sequence of agent steps, tool calls, approvals, and state transitions.
ToolCallEval	Validates that tools were called with expected schemas, risk levels, and order.
SafetyEval	Checks prompt injection, PHI leaks, false standards claims, unauthorized tool calls.
Scorecard	Human-readable pass/fail summary for demos and CI.
TraceDiff	Explains behavior changes between current run and golden trace.

17.2 Initial scenarios

Scenario	Purpose	Expected outcome
mri_lumbar_spine_success	Happy path with complete evidence.	Packet submitted to mock payer; approved or pended fixture result.
mri_missing_neuro_exam	Evidence gap case.	Agent blocks before packet readiness and asks for missing evidence or human override.
mri_more_info_loop	Payer requests additional documentation.	Case returns to documentation/evidence workspace without losing prior packet history.
mri_denial_explain	Denied case with reason.	Agent explains denial administratively and drafts next-step options.
specialty_drug_prior_auth	Second service-line proof.	Same engine handles different questionnaire and rule pack.
prompt_injection_evidence	Malicious text embedded in evidence fixture.	Agent ignores injected instructions and does not call unauthorized tools.

17.3 Golden trace checks

- Plan includes the required high-level steps for the scenario.

- Only allowed agents and tools are used.
- Tool inputs validate against schemas.
- Write/submit operations are approval-gated.
- Evidence mappings cite evidence IDs and requirement IDs.
- Missing-data reasons are explicit.
- Final case state matches expected outcome.
- No false claims of production conformance or PHI readiness appear in outputs.

18. Safety, policy, and governance model

The safety model should be architectural, not just prompt-based. The system should assume model outputs are untrusted until validated by schemas, policy checks, tool allowlists, and human approvals. Generative AI risk guidance from NIST and OWASP supports treating model behavior, data privacy, information integrity, supply chain, prompt injection, and excessive agency as first-class design concerns [R13][R14][R15].

18.1 Policy categories

Policy	Behavior
Synthetic data guard	Default mode rejects or warns on real-looking PHI and states synthetic-only posture.
Tool allowlist	Each agent can call only assigned tools.
Risk-level approval	write, submit, and external tool calls are approval-gated.
Medical-advice boundary	Agents may explain administrative requirements but not make clinical treatment decisions.
Standards-claim guard	Blocks or rewrites claims such as certified, production-ready, PHI-ready, live payer integrated.
Evidence provenance	Every evidence item and mapping has source, type, checksum/pointer, and synthetic/user/external marker.
Prompt-injection defense	Treats evidence text and external artifacts as data, not instructions.
Audit completeness	Case-changing actions require actor, tool, timestamp, before/after, and correlation ID.

18.2 Approval gates

Action	Default behavior
Read case/evidence/audit	Allowed for authorized local user/agent.
Draft questionnaire or packet preview	Allowed, trace required.
Save questionnaire response	Approval in reviewed-live mode or UI staff action.
Attach evidence	Approval in reviewed-live mode or UI staff action.
Change case status	Approval or explicit UI action required.
Submit mock PAS packet	Always approval.
Call external service	Out of v0.1; future always approval and policy-reviewed.

18.3 ComplianceBoundaryAgent checks

- Does the action use only synthetic data?
- Is the tool allowed for this agent and mode?
- Is the tool call schema-valid?
- Does the action require human approval?
- Does the output imply clinical judgment beyond administrative workflow?
- Does the output make unsupported standards or production claims?
- Can the audit trace explain evidence, tool, and decision provenance?

19. Technical stack and build sequencing

The stack should remain TypeScript-first for product, workflow, standards adapters, tool contracts, MCP, and UI. Python can remain a future sidecar for document intelligence and eval-heavy extraction. The key change is that the runtime/tool layers become first-class packages instead of incidental API handlers.

Layer	Recommended direction	Open-source build target now	Why
Frontend	Next.js + React + TypeScript + shadcn/Tailwind.	Agent cockpit plus prior-auth screens.	Strong product UI for queues, timelines, approval, evidence, evals.
Agent runtime	Custom Doctor Runtime.	Build from scratch with replay/live-local/reviewed-live modes.	Core differentiator and learning goal.
Tool runtime	Custom Doctor ToolNet.	Registry, broker, schemas, policy, trace records.	Safe tools are the backbone of agent behavior.
MCP	TypeScript MCP server over ToolNet.	stdio first, HTTP later.	External agent interoperability without losing internal control.
API	TypeScript product API and standards routes.	Keep local API, add agent-run endpoints and standards gateway.	Integrates UI, agents, tools, and fixtures.
State store	SQLite local; Postgres production path.	SQLite for local reproducibility.	Enough for OSS; migration path is documented.
Workflow engine	Custom semantics first; Temporal path later.	Pause/resume, checkpoints, retries, approvals.	Prior auth has waits and human tasks.
FHIR substrate	Adapter boundary; Medplum/HAPI later.	FHIR-shaped fixtures behind repository interface.	Avoid hardwiring to fixture files.
Document intelligence	Python sidecar later.	Evidence metadata and mapping now.	Extraction should wait until core workflow is stable.
Observability	Structured audit now; OpenTelemetry/Langfuse later if AI is live.	Trace tables and eval artifacts.	Separate system failures from model/tool behavior.

19.1 Where to spend effort next

Priority	Work	Reason
Highest	M0/M1 repo reset and ToolNet.	Without typed tools, agents become unsafe and hard to test.
Highest	M2 Doctor Runtime.	The project goal is to build an agentic system from scratch.
High	M3 prior-auth agent team.	Creates the flagship demo.
High	M4 agent cockpit.	Makes the system legible to product and engineering audiences.
High	M5 MCP server.	Shows modern agent interoperability.
Medium	M6 durable workflow semantics.	Turns demo into long-running case architecture.
Medium	M7 standards gateway.	Preserves healthcare standards credibility.
Medium	M8 eval harness.	Makes behavior regression-testable.
Later	Document intelligence.	Powerful, but only after core workflow and evidence model are stable.
Later	Real integration pilot.	Requires security posture, agreements, and constrained scope.

20. Roadmap and delivery plan

The new roadmap should start with architecture reset and runtime/tool foundations. The previous M8-M14 standards roadmap remains valuable, but it should be nested inside the agentic roadmap rather than driving the entire project alone.

Milestone	Deliverables	Exit criteria
M0: Agentic strategy reset	README direction, architecture docs, repo map, AGENTS.md, product vision, claim language.	Project is clearly positioned as Doctor Agent OS with prior auth as first app.
M1: Doctor ToolNet	Tool registry, broker, schemas, policies, risk levels, trace records, initial tools.	Tools are discoverable, validated, permissioned, and logged.
M2: Doctor Runtime	AgentSpec, AgentRunner, RunState, TaskPlan, Handoff, ApprovalGate, TraceRecorder, replay mode.	Agent can plan, call allowed tools, pause for approval, and record trace.
M3: Prior-auth agent team	Orchestrator, intake, requirements, documentation, evidence, packet, payer followup, compliance agents.	MRI case moves from intake to packet preview with visible agent trace.
M4: Agent cockpit UI	Run timeline, tool viewer, approval panel, evidence board, packet preview, audit trace.	A user can understand what happened and approve/reject risky steps.
M5: Doctor MCP	stdio MCP server, resources, prompts, safe tools, MCP inspector docs.	External MCP client can discover and call safe read tools.
M6: Durable workflow semantics	SQLite run store, pause/resume, retries, idempotency, approval persistence.	Agent run survives pause/restart and can replay.
M7: Standards Gateway	SMART, CDS Hooks/CRD, DTR package, PAS	Standards-shaped boundaries map clearly to internal tools.

Milestone	Deliverables	Exit criteria
	Claim/\$submit, OperationOutcome-style errors, conformance matrix.	
M8: Doctor Evals	Golden traces, scorecards, tool-call evals, safety evals, prompt-injection fixtures.	CI can catch behavior regressions.
M9: Reusable workflow proof	Second payer, second service line, differentiated questionnaires and payer outcomes.	Same engine handles at least two payer/service-line combinations.
M10: Production-path docs	FHIR adapter, auth/security/audit, observability, deployment, integration path.	The gap map is specific and credible.

20.1 MVP definition

MVP capability	Included?
Custom agent runtime	Yes.
Custom tool registry/broker	Yes.
Custom MCP server	Yes, stdio first.
Multi-agent prior-auth workflow	Yes.
Human approval gates	Yes.
Agent trace UI	Yes.
Synthetic FHIR context	Yes.
CRD/DTR/PAS-shaped tools	Yes.
Mock payer	Yes.
Golden trace eval	Yes.
Real PHI	No.
Real EHR or payer integration	No.
Production conformance claim	No.

21. Definition of done

Area	Done means
Local run	A developer can clone, install, seed, run, and open the agentic prior-auth demo with documented commands.
Agent run	One polished case run works from order context to payer determination, with visible plan, handoffs, tools, approvals, and trace.
Tool runtime	Every agent action goes through ToolNet with schema validation, risk level, policy check, and tool-call record.
MCP	Safe resources, prompts, and read tools are exposed through MCP and documented.
Standards harness	SMART/CRD/DTR/PAS/FHIR-shaped boundaries are visible in routes, docs, tests, and artifacts.
Conformance honesty	The conformance/gap matrix is visible, specific, and maintained.
Operational realism	Queue, aging, more-info, denial, approval, audit timeline, and metrics are visible.
Evaluations	Golden traces, tool-call evals, and safety evals run locally and in CI.
Reusability	At least two payer/service-line combinations use the same engine.
Safety posture	Synthetic data only; no claim of PHI readiness, clinical decision automation, or certified conformance.

22. Production readiness gap map

Production requirement	Current / OSS posture	Path forward
Real SMART App Launch	SMART-shaped metadata and local context only.	Add sandbox EHR registration, auth adapter tests, token/context handling, vendor notes.
CDS Hooks CRD conformance	CRD-style fixture services.	Validate against CRD IG expectations and Inferno test

Production requirement	Current / OSS posture	Path forward
		paths; document unsupported hooks.
DTR production behavior	Synthetic questionnaire packages.	Add deeper SDC support, dependency resolution, prepopulation tests, payer package adapters.
PAS production submission	Mock Claim/\$submit and deterministic packet.	Add payer connector boundary, endpoint discovery, status inquiry, transport security, payer mappings.
FHIR data plane	FHIR-shaped fixtures and repositories.	Move to FHIR adapter; add Medplum/HAPI-backed implementation.
Durable case workflow	Local run store and lifecycle semantics.	Introduce Temporal or equivalent durable workflow with timers, retries, signals, human tasks.
Agent production safety	Local supervised agents and synthetic evals.	Add stronger auth, data minimization, redaction, prompt-injection hardening, model/provider evals.
Enterprise authz	Local/demo identity.	Add OIDC/Keycloak and fine-grained org/tenant/case permissions.
PHI-safe logging	Synthetic-only posture.	Add redaction policies, secure secrets, access logs, retention/deletion controls.
Document intelligence	Evidence metadata and fixture mapping.	Add OCR/parsing, confidence, human acceptance, provenance, extraction evals.
Observability	Structured traces/audit/eval artifacts.	Add OpenTelemetry traces/metrics and AI observability only when live AI is used.
Deployment hardening	Local dev environment.	Add Docker hardening, k8s manifests, CI/CD, env separation, health checks, monitoring.

23. Risks and mitigation plan

Risk	Why it matters	Mitigation
Scope creep	The project can drift into full RCM, clinical AI, payer portal, claims adjudication, or broad agent platform.	Keep prior auth as the flagship wedge and tie every milestone to the case lifecycle.
Overclaiming conformance	Healthcare stakeholders distrust vague compliance claims.	Maintain explicit conformance/gap matrix and careful claim language.
Agent overreach	Agents may appear to make clinical or submission decisions autonomously.	Use tool risk levels, approval gates, policy checks, and medical-advice boundary.
Prompt injection	Evidence or external content can include malicious instructions.	Treat content as data, use allowlisted tools, block unauthorized actions, run safety evals.
MCP tool exposure	MCP can expose powerful tools to external hosts.	Expose read-only tools first; approval-gate writes/submits; document security posture.
Runtime complexity	Building from scratch can slow product progress.	Implement minimal runtime first: plan, tools, approval, trace, replay. Avoid framework over-engineering.
Weak evaluation	Agent behavior can regress silently.	Golden traces and scorecards must be part of CI.
FHIR/DTR renderer edge cases	Complex forms can break UX and standards behavior.	Renderer abstraction, deterministic fixtures, conformance docs, and test cases.
Document extraction errors	Bad extraction can poison evidence mapping.	Delay extraction; require confidence, provenance, citations, human acceptance.
PHI/compliance risk	Real health data creates legal and operational obligations.	Default synthetic-only; block PHI claims; add production security only when ready.

24. Repository documentation and external-facing artifacts

Artifact	Purpose	Key content
README.md	Public entry point.	New positioning, quickstart, demo flow, screenshots, synthetic-data warning, standards-shaped disclaimer.
AGENTS.md	Agent development guide.	Agent roles, allowed tools, safety rules, output schemas, repo instructions.
docs/architecture/doctor-runtime.md	Runtime source of truth.	AgentSpec, RunState, handoffs, approvals, traces, modes.
docs/architecture/toolnet.md	Tool architecture.	Registry, broker, schemas, risk levels, approval policy, trace records.
docs/architecture/mcp-server.md	MCP boundary.	Resources, prompts, tools, transports, safety posture.
docs/standards/conformance-matrix.md	Truth table for standards claims.	SMART/CRD/DTR/PAS/FHIR/Inferno/MCP/security rows with support and gaps.
docs/demo/story-flow.md	Demo script.	Order -> agent plan -> requirements -> documentation ->

Artifact	Purpose	Key content
		evidence -> approval -> packet -> payer -> audit.
docs/evals/golden-traces.md	Evaluation guide.	Scenario definitions, trace matching, scorecards, safety evals.
docs/security/synthetic-data-and-phi.md	Safety posture.	Synthetic-only policy, PHI warnings, logging principles, approval gates.
docs/roadmap.md	Milestone planning.	M0-M10 sequencing with exit criteria.

24.1 README language to use

Doctor Agent OS is a synthetic-data, open-source agentic reference system for healthcare administrative workflows. Its first application, Open Prior Auth Agent Workbench, demonstrates a supervised prior authorization agent run across requirement discovery, DTR-style documentation, evidence mapping, human approval, PAS-style packet assembly, mock payer response tracking, and operational dashboards. It is not a production payer connector, not a certified Da Vinci implementation, and not intended for real PHI. Its purpose is to make the production architecture understandable, testable, and extensible.

25. Final recommendation

Build Doctor Agent OS as the new strategic direction and keep Open Prior Auth Agent Workbench as the first flagship workflow. The project should now be judged by whether it can demonstrate a trustworthy supervised agent system: custom runtime, custom tool layer, MCP boundary, standards-shaped healthcare adapters, human approval gates, auditability, and deterministic evaluations.

The immediate objective should not be production SMART, real payer transport, real EHR integration, real PHI handling, or formal Da Vinci certification. Those remain production north-star items. The immediate objective is to build an impressive and honest open-source agentic reference system that makes the future production architecture tangible.

The best next sequence is M0 strategy/repo reset, M1 Doctor ToolNet, M2 Doctor Runtime, M3 prior-auth agent team, M4 agent cockpit, M5 MCP server, M6 durable workflow semantics, M7 standards gateway, and M8 evaluation harness. This sequence proves the new direction from first principles while preserving the current domain and standards credibility.

Source of truth: Final source-of-truth statement:

Doctor Agent OS aims to be the best open-source, synthetic-data-only agentic reference system for healthcare administrative workflows, beginning with provider-side prior authorization. It demonstrates how supervised agents, typed tools, human approvals, standards-shaped healthcare adapters, deterministic fixtures, and audit/eval traces can work together to make prior authorization understandable, testable, and extensible. It does not claim production readiness, PHI readiness, live payer/EHR integration, or certification.

26. References

[R0] Open Prior Auth Workbench revised strategy report, uploaded May 2026. Used as the base strategy document for this agentic update.

[R1] CMS. Interoperability and Prior Authorization Final Rule CMS-0057-F. <https://www.cms.gov/priorities/burden-reduction/overview/interoperability/policies-regulations/cms-interoperability-prior-authorization-final-rule-cms-0057-f>

[R2] HL7 Da Vinci. Coverage Requirements Discovery Implementation Guide. <https://build.fhir.org/ig/HL7/davinci-crd/>

[R3] HL7 Da Vinci. Documentation Templates and Rules Implementation Guide and questionnaire-package operation. <https://build.fhir.org/ig/HL7/davinci-dtr/> and <https://build.fhir.org/ig/HL7/davinci-dtr/en/OperationDefinition-questionnaire-package.html>

[R4] HL7 Da Vinci. Prior Authorization Support Implementation Guide. <https://build.fhir.org/ig/HL7/davinci-pas/>

[R5] HL7 SMART App Launch Implementation Guide. <https://hl7.org/fhir/smart-app-launch/>

[R6] Inferno on HealthIT.gov. Test Kits, including Da Vinci CRD, DTR, PAS, and SMART. <https://inferno.healthit.gov/test-kits/>

[R7] ONC / HealthIT.gov. Enhancing Healthcare Interoperability: launching the Da Vinci PAS Test Kit. <https://healthit.gov/blog/interoperability/enhancing-healthcare-interoperability-launching-the-davinci-prior-authorization-support-pas-test-kit/>

[R8] Model Context Protocol. Specification and architecture. <https://modelcontextprotocol.io/specification/> and <https://modelcontextprotocol.io/docs/learn/architecture>

[R9] Model Context Protocol. Tools specification. <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>

[R10] Anthropic. Introducing the Model Context Protocol. <https://www.anthropic.com/news/model-context-protocol>

- [R11] OpenAI. Agents SDK guide. <https://developers.openai.com/api/docs/guides/agents>
- [R12] OpenAI Agents SDK docs: agents, tools, tracing, MCP. <https://openai.github.io/openai-agents-python/agents/>
- [R13] NIST. AI Risk Management Framework: Generative AI Profile (NIST AI 600-1). <https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence>
- [R14] OWASP. Top 10 for Large Language Model Applications. <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- [R15] OWASP GenAI Security. Prompt Injection and Excessive Agency risk pages. <https://genai.owasp.org/llmrisk/llm01-prompt-injection/> and <https://genai.owasp.org/llmrisk/llm06-excessive-agency/>
- [R16] Temporal. Durable execution/workflows. <https://github.com/temporalio/temporal>
- [R17] Medplum. Open-source healthcare and FHIR platform. <https://www.medplum.com/>
- [R18] HAPI FHIR JPA Server Starter. <https://github.com/hapifhir/hapi-fhir-jpaserver-starter>
- [R19] OpenFGA. Fine-grained authorization. <https://openfga.dev/>
- [R20] Keycloak. Open-source identity and access management. <https://github.com/keycloak/keycloak>
- [R21] Synthea. Synthetic patient population simulator. <https://github.com/synthetichealth/synthea>